SB SECURITY

Shift V2

Security Review



March 21, 2026 - March 26, 2026

Contents

1. About SBSecurity 4
2. Disclaimer 4
3. Risk classification 4
4. Executive Summary 5
5. Findings 6

Medium severity 9

[M-01] Incorrect share price calculation when collecting performance fee 9
[M-02] First epoch settling will overcharge maintenance fee 10
[M-03] FastRedeem feature can be permanently Dos'd 11
[M-04] Attacker can inflate the share price by delaying deposit calls 12
[M-05] ERC20Burnable::burn/burnFrom Not Overridden 13
[M-06] Unchecked _controller in Hub::_requireOwnerOrOperator enables deposit and redeem griefing 15
[M-07] Malicious User Can Grief Fast Redeems by Occupying the Cap Until cutoffEpoch and Re-requesting Before lockEpoch 17

Low severity 20

[L-01] Rate limit applied to gross amount instead of net amount after dust removal 20
[L-02] Allocator Grace Period Off-by-One: Only 1 Skip Instead of 2 Before Price Override 22
[L-03] Unbounded deposit epoch Iterations make gas costs high for long-term depositors 23
[L-04] Price-Zero Cascade: Maintenance Fee Drives Share Price to Zero, Enabling Sentinel Inflation and Delta Bypass 24
[L-05] Fast redeem fee read from storage at claim time instead of request 27
[L-06] First Depositor can inflate hwm with low supply resulting in lost performance fee for the protocol 29
[L-07] requestFastRedeem Incorrectly Blocked in Deprecated Vault State 30
[L-08] requestRedeemStatus Returns False PastCutoff for Normal Redeem Cycles 31
[L-09] pendingDepositRequest Returns Non-Zero After Epoch Settlement 33
[L-10] supportsInterface Does Not Return True for EIP-165 Interface ID 34
[L-11] setAllocator Does Not Reset allocatorActive Flag 35
[L-12] Decimal mismatch between syShare and collateral 36
[L-13] requestFastRedeemStatus Returns Ok When Remaining Cap Is Below s_minRedeem 37
[L-14] s_minRedeem can be enforced in redeem/withdraw functions to prevent users from holding dust amounts in s_pendingRedeems and possible dust accumulation in pool 38
[L-15] HWM stale when performance fee is zero, retroactive fee collection - Formal Verification 40

Informational severity 41

[I-01] Redeem status views defined in syVault instead of SyVaultRedeem 41
[I-02] Misleading error name in cancel redeem functions 42
[I-03] Incorrect NatSpec for s_lastFeeCollectionEpoch 43

[I-04] setMaxTvl Should Not Allow Setting s_maxTVL Below the Current TVL 44
[I-05] set_fast_redeem_fee_bps allows 100% fee, leading to division by zero in withdraw and total fund loss in redeem 45
[I-06] Deposit epoch consumption loops use continue instead of break on unsettled epochs 46
[I-07] claimableFastRedeemRequest Returns An _assets Value That withdraw Cannot Accept 48

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Twitter @Slavcheww.

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security-related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1. Impact

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2. Likelihood

- **High** - almost **certain** to happen, easy to perform, or highly incentivized.
- **Medium** - only **conditionally possible**, but still relatively likely.
- **Low** - requires specific state or **little-to-no incentive**.

3.3. Action required for severity levels

- High - **Must** fix (before deployment if not already deployed).
- Medium - **Should** fix.
- Low - **Could** fix.

4. Executive Summary

A time-boxed security review of the **SHIFT-NebulaLabs/shift-core-v2** repository was conducted by **SBSecurity**. The review was performed by **MonteCristo**, **Codex, blckhv, Velev, Slavcheww** who collectively identified **28 issues** in total.

Overview

Project	Shift V2
Commit Hash	bfcd9bf2a20a4c6e07f921bb03eb1cb1d280976c
Resolution	537d2d8b3eadc1a93f118cb850bb232d9d7470c6
Timeline	March 21, 2026 - March 26, 2026

Scope

syVault.sol, abstracts/SyVaultRedeem.sol, abstracts/SyVaultEpoch.sol, abstracts/SyVaultBase.sol, Hub.sol, AdminGateway.sol, AddressRegistry.sol, LiquidityPool.sol, syShare.sol, crosschain/syOFTAdapter.sol, crosschain/sySharePeer.sol, types/Constants.sol, types/Errors.sol, types/Types.sol

Post Audit Condition

The codebase demonstrates solid development practices. However, 7 Medium and 14 Low severity issues were identified, along with several informational findings. A follow-up audit is recommended to ensure all concerns are properly mitigated.

5. Findings

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	7
Low Risk	15
Informational Risk	7

ID	Title	Severity	Resolution
[M-01]	Incorrect share price calculation when collecting performance fee	Medium	Fixed
[M-02]	First epoch settling will overcharge maintenance fee	Medium	Fixed
[M-03]	`FastRedeem` feature can be permanently Dos'd	Medium	Fixed
[M-04]	Attacker can inflate the share price by delaying deposit calls	Medium	Fixed
[M-05]	ERC20Burnable::burn/burnFrom Not Overridden	Medium	Fixed
[M-06]	Unchecked `_controller` in `Hub::_requireOwnerOrOperator` enables deposit and redeem griefing	Medium	Fixed
[M-07]	Malicious User Can Grief Fast Redeems by Occupying the Cap Until `cutoffEpoch` and Re-requesting Before `lockEpoch`	Medium	Fixed
[L-01]	Rate limit applied to gross amount instead of net amount after dust removal	Low	Fixed
[L-02]	Allocator Grace Period Off-by-One: Only 1 Skip Instead of 2 Before Price Override	Low	Fixed
[L-03]	Unbounded deposit epoch Iterations make gas costs high for long-term depositors	Low	Fixed
[L-04]	Price-Zero Cascade: Maintenance Fee Drives Share Price to Zero, Enabling Sentinel Inflation and Delta Bypass	Low	Fixed

[L-05]	Fast redeem fee read from storage at claim time instead of request	Low	**Fixed**
[L-06]	First Depositor can inflate `hwm` with low supply resulting in lost `performance fee` for the protocol	Low	Acknowledged
[L-07]	`requestFastRedeem` Incorrectly Blocked in Deprecated Vault State	Low	**Fixed**
[L-08]	`requestRedeemStatus` Returns False `PastCutoff` for Normal Redeem Cycles	Low	**Fixed**
[L-09]	`pendingDepositRequest` Returns Non-Zero After Epoch Settlement	Low	**Fixed**
[L-10]	`supportsInterface` Does Not Return True for EIP-165 Interface ID	Low	**Fixed**
[L-11]	`setAllocator` Does Not Reset `allocatorActive` Flag	Low	**Fixed**
[L-12]	Decimal mismatch between syShare and collateral	Low	Acknowledged
[L-13]	`requestFastRedeemStatus` Returns `Ok` When Remaining Cap Is Below `s_minRedeem`	Low	**Fixed**
[L-14]	`s_minRedeem` can be enforced in `redeem`/`withdraw` functions to prevent users from holding dust amounts in `s_pendingRedeems` and possible dust accumulation in pool	Low	**Fixed**
[L-15]	HWM stale when performance fee is zero, retroactive fee collection - Formal Verification	Low	**Fixed**
[I-01]	Redeem status views defined in `syVault` instead of `SyVaultRedeem`	Info	**Fixed**
[I-02]	Misleading error name in cancel redeem functions	Info	**Fixed**
[I-03]	Incorrect NatSpec for s_lastFeeCollectionEpoch	Info	**Fixed**
[I-04]	`setMaxTvl` Should Not Allow Setting `s_maxTVL` Below the `Current TVL`	Info	Acknowledged
[I-05]	`set_fast_redeem_fee_bps` allows 100% fee, leading to division by zero in `withdraw` and total fund loss in `redeem`	Info	**Fixed**
[I-06]	Deposit epoch consumption loops use `continue` instead of `break` on unsettled epochs	Info	**Fixed**

[I-07]	`claimableFastRedeemRequest` Returns An `_assets` Value That `withdraw` Cannot Accept	Info	**Fixed**

Medium severity

[M-01] Incorrect share price calculation when collecting performance fee

Severity: Medium Risk

Description: When performance fee is non-zero, fee share will be minted to fee recipient according to dilution model:

VaultEpoch.sol

```
128:         uint256 feeSharesMinted;
129:         if (_price > hwm && s_performanceFeeBps > 0) {
130:             if (supply > 0) {
131:                 // Dilution model: feeShares = (price - Hwm) * supply * feeBps / (price *
BASIS_POINTS)
132:@>               feeSharesMinted = ((_price - hwm) * supply * s_performanceFeeBps)
133:                     / (_price * BASIS_POINTS);
134:                 if (feeSharesMinted > 0) i_share.mint(s_feeRecipient, feeSharesMinted);
135:             }
136:             s_hwm = _price;
137:             emit PerformanceFeeCollected(_epoch, _price, hwm, feeSharesMinted);
138:         }
```

However, price is not deducted to reflect dilution.

As a result, TVL will be evaluated higher in the following line (i.e. performance fee is double counted):

syVault.sol

```
141:             uint256 tvl = (s_sharePrice * i_share.totalSupply()) / SHARE_PRICE_PRECISION;
```

Impact:

Share price is not correctly calculated. This will lead to the following impacts:

- Max TVL check is wrong in `SyVault.requestDeposit`. TVL is over-estimated and `maxDeposit` will be under-estimated
- Offchain TVL calculation will be over-estimated

Recommendation:

Adjusted `_price` after dilution fee share is minted

Resolution: Fixed

[M-02] First epoch settling will overcharge maintenance fee

Severity: Medium Risk

Description: `s_nextEpochToSettle` is set to 0 by default so first `epochSettle` call is eligible for maintenance fee collection:

SyVaultEpoch.sol

```
141:         if (s_maintenanceFeeInterval > 0 && s_maintenanceFeeBps > 0
142:@>           && (_epoch % s_maintenanceFeeInterval) == 0)
143:         {
144:             uint256 priceBefore  = _price;
145:             uint256 feeDeduction = (_price * s_maintenanceFeeBps * s_maintenanceFeeInterval)
146:                 / (BASIS_POINTS * EPOCHS_PER_YEAR);
147:             _price = _price > feeDeduction ? _price - feeDeduction : 0;
148:
```

The fee is charged for whole `s_maintenanceFeeInterval` but the actual elapsed epoch is 1.

So the maintenance fee is overcharged by `s_maintenanceFeeInterval - 1` times.

Impact:

Maintenance fee is overcharged in the first epoch settling.

Recommendation:

Skip maintenance fee charging if `_epoch = 0`

Resolution: Fixed

[M-03] `FastRedeem` feature can be permanently Dos'd

Severity: Medium Risk

Description: If a user calls `requestFastRedeem()` and then cancels it using `cancelFastRedeem()`, and no other user calls `requestFastRedeem()` until the cycle cutoff, the cycle state will be:

```
s_currentFastCycleId != 0; //since a user requestedFastredeem atleast once for that cycle
currentEpoch() >= s_redeemCycles[s_currentFastCycleId].cutoffEpoch; // time has passed cutoff
cycle.shares == 0; //since the user cancelled his request
```

This creates a situation where a new user cannot call `requestFastRedeem()` because the previous cycle remains unsettled.

```
function _openOrReuseCycle(CycleType _cycleType) internal returns (uint256 cycleId) {
    if (_cycleType == CycleType.Fast) {
        if (s_currentFastCycleId == 0 || currentEpoch() >=
s_redeemCycles[s_currentFastCycleId].cutoffEpoch) {
            // Only one unsettled fast cycle at a time — Allocator handles one per day.

            if (s_currentFastCycleId != 0 && !s_redeemCycles[s_currentFastCycleId].settled) {
                revert SyVault__FastCycleNotSettled();  <--------@ Reverts here
            }
            ...
            ..
            .
        }
    }
```

Additionally, the allocator's call to `settleRedeemBatch()` also reverts because `cycle.shares == 0`

```
function settleRedeemBatch(uint256 _cycleId, uint256 _returnedAmount) external onlyAllocator nonReentrant {
    RedeemCycle storage cycle = s_redeemCycles[_cycleId];
    if (cycle.shares == 0) revert SyVault__NoPendingRequest(); <-------@ Reverts here
    ..
}
```

As a result, the `fastRedeem` feature becomes permanently unusable.

Impact:

The `fastRedeem` functionality can be permanently DoS'd, preventing users from accessing this feature.

Recommendation:

The `settleRedeemBatch()` function should settle the cycle even if `cycle.shares == 0`, also add a check that if `cycle.shares == 0`, then `returnedAmount` is also `0` to ensure a clean settlement.Only burn shares when `cycle.shares > 0` to avoid unnecessary operations or errors.

Resolution: Fixed

[M-04] Attacker can inflate the share price by delaying deposit calls

Severity: Medium Risk

Description: The following is offchain logic to calculate `sharePrice`:

```
TVL        = Equity + investableLiquidity()
sharePrice = (TVL × 1e18) / effectiveSupply()
```

An attacker can inflate share price by increasing TVL while keeping `effectiveSupply` unchanged. Consider the following attack path:

- Attacker requests deposit of amount △

- Epoch is settled.

- TVL will increase by △ (i.e. `s_totalPendingDeposits` is decreased by △)

- Attacker does not call mint or deposit for 1 epoch

- `effectiveSupply` will not be increased because attacker's share is not minted yet

- Next epoch is settled

- New share price will be `((TVL + △) × 1e18) / effectiveSupply()`, increased by `△ × 1e18 / effectiveSupply()`

- Now the attacker calls mint/deposit

- In this case, minted amount is determined by old `sharePrice`, not by the new one

- This effectively means the attacker has an instant profiti

Impact:

`sharePrice` is incorrectly estimated.

In the worst case, if `cycleRate` follows the similar logic to `sharePrice`, the attacker can realize the profit by requesting redeem. The attacker can repeat the attack indefinitely.

Recommendation:

- In `_onEpochSettle`, mint a buffer share to cover `pendingAssets`

- When users mint or deposit, burn corresponding amount from buffer to hold `totalSupply` invariant

Resolution: Fixed

[M-05] ERC20Burnable::burn/burnFrom Not Overridden

Severity: Medium Risk

Description: `SyShare` inherits OpenZeppelin's `ERC20Burnable`, which exposes two public functions: `burn(uint256 amount)` (1-parameter) and `burnFrom(address account, uint256 amount)`. The vault defines its own `burn(address from, uint256 amount)` (2-parameter, `onlyVault`), but because this has a different function selector it does NOT override `ERC20Burnable.burn(uint256)`. Both inherited burn functions remain publicly callable by any address.

```
// syShare.sol:17 — inherits both ERC20 and ERC20Burnable
contract SyShare is ERC20, ERC20Burnable, ISyShare {

// The vault's 2-param burn (onlyVault) does NOT shadow the inherited 1-param public burn:
function burn(address _from, uint256 _amount) external onlyVault { ... }

// ERC20Burnable.burn(uint256 amount) — public, not overridden, still callable by anyone
// ERC20Burnable.burnFrom(address, uint256) — public, not overridden, still callable by anyone
```

When an external actor calls `syShare.burn(N)`, `totalSupply` decreases by `N` but `s_totalLockedShares` in the vault is unchanged. This breaks the vault's `effectiveSupply()` denominator:

```
// SyVaultRedeem.sol — effectiveSupply used by keeper's NAV formula
function effectiveSupply() external view returns (uint256) {
    uint256 supply = i_share.totalSupply();
    uint256 locked = s_totalLockedShares;
    return supply > locked ? supply - locked : 0;
}
```

Because `effectiveSupply()` is the denominator in the keeper's NAV formula, burning shares externally distorts the ratio between outstanding supply and locked shares — the keeper computes a share price against fewer free-floating shares than actually back the vault's assets, enabling a holder to manipulate the reported price upward or downward to their advantage.

Impact:

Any holder of `syShare` can call `burn(amount)` to reduce their own balance AND the global `totalSupply`, permanently distorting the keeper's NAV calculation and `totalAssets()` view. In extremis, a griever holding a small position can drive `effectiveSupply` to zero, DoS-ing epoch settlement. A user who acquired shares cheaply could use this to manipulate performance fee computation.

Recommendation:

Override both `burn(uint256)` and `burnFrom(address, uint256)` in `SyShare` to revert unconditionally, preventing any external burn path. The vault's 2-parameter `burn(address, uint256)` with `onlyVault` is the only legitimate burn path and it calls the internal `_burn` directly, so no functionality is lost.

```
// Add to SyShare:
function burn(uint256) public pure override { revert SyShare__NotVault(); }
function burnFrom(address, uint256) public pure override { revert SyShare__NotVault(); }
```

Resolution: Fixed

[M-06] Unchecked `_controller` in `Hub::_requireOwnerOrOperator` enables deposit and redeem griefing

Severity: Medium Risk

Description: `Hub::_requireOwnerOrOperator` ignores the `_controller` parameter entirely (`address /* _controller */`) and only validates that `msg.sender` is `_owner` or an approved operator of `_owner`. Since `_controller` is unchecked, any user can pass an arbitrary victim address as `_controller` while setting themselves as `_owner` to pass auth.

Hub.sol#L163-L169

```
function _requireOwnerOrOperator(uint256 _vaultId, address /* _controller */, address _owner) internal view
{
    if (msg.sender != _owner) {
        if (!ISyVault(_getSyVault(_vaultId)).isOperator(_owner, msg.sender)) {
            revert Hub__NotAuthorized();
        }
    }
}
```

This affects three entry points that pass separate `_controller` and `_owner` to `_requireOwnerOrOperator`:

- `Hub::requestDeposit` - attacker deposits dust under the victim's `_controller`, binding `s_depositRequestOwner[victim][epoch]` to the attacker. The victim's subsequent deposit in the same epoch reverts with `SyVault__InvalidOwner` at the `else if` branch. The victim cannot remedy this within the epoch and must wait for the next one (1 hour). The attacker's dust is a net loss - they cannot cancel or claim it, and the victim receives it as free shares after settlement.
- `Hub::requestRedeem` / `Hub::requestFastRedeem` - attacker sets `s_activeRedeemCycleId[victim]` to a non-zero value, blocking the victim's redeem with `SyVault__RedeemRequestAlreadyPending`. This variant is less severe: the attacker must hold actual shares, and the victim can self-remedy by calling `Hub::cancelRedeemRequest` (passes auth since `_controller == victim`), which returns the attacker's shares and clears the block.

syVault.sol#L172-L178

```
if (!s_epochPushed[_controller][requestId]) {
    s_controllerDepositEpochs[_controller].push(requestId);
    s_epochPushed[_controller][requestId] = true;
    s_depositRequestOwner[_controller][requestId] = _owner;
} else if (s_depositRequestOwner[_controller][requestId] != _owner) {
    revert SyVault__InvalidOwner();
}
```

Impact:

Any user can block any other user's deposits for the current epoch (1 hour) at the cost of `s_minDeposit` worth of assets (or 1 wei if no minimum is set). The attack is repeatable every epoch. Redeem blocking is also possible but self-remediable by the victim.

Proof Of Concept:

1. Victim submits `Hub::requestDeposit(vaultId, 1000e6, victim, victim)` to the mempool

2. Attacker frontruns with `Hub::requestDeposit(vaultId, 1, victim, attacker)` - passes auth since `msg.sender == _owner == attacker`

3. `s_depositRequestOwner[victim][currentEpoch]` is set to `attacker`

4. Victim's tx reverts with `SyVault__InvalidOwner` because `attacker != victim`

Recommendation:

Validate that `msg.sender` is the `_controller` or an approved operator of the `_controller` in `Hub::_requireOwnerOrOperator`. This single fix addresses all three affected entry points.

Resolution: Fixed

[M-07] Malicious User Can Grief Fast Redeems by Occupying the Cap Until `cutoffEpoch` and Re-requesting Before `lockEpoch`

Severity: Medium Risk

Description: The `requestFastRedeem` function enforces a check to ensure that any new redeem request does not exceed `capShares`. If the cap is exceeded, the transaction reverts.

This design enables a malicious user to indefinitely block the redeem cap up to `cycle.cutoffEpoch`. The attacker can then cancel their request and immediately submit a new fast redeem request before `cycle.lockEpoch`. As a result, legitimate users are unable to submit fast redeem requests, and the admin is also unable to call `settleRedeemBatch` for that cycle since the `lockEpoch` has not yet been reached.

SyVaultRedeem.sol:138-172

```
function requestFastRedeem(uint256 _shares, address _controller, address _owner)
    external
    onlyHub
    whenRedeemsNotPaused
    returns (uint256 requestId)
{
    if (!s_fastRedeemEnabled) revert SyVault__FastRedeemDisabled();
    if (_shares == 0) revert SyVault__ZeroAmount();
    if (s_minRedeem > 0 && _shares < s_minRedeem) revert SyVault__BelowMinRedeem();
    if (s_activeRedeemCycleId[_controller] != 0) revert SyVault__RedeemRequestAlreadyPending();

    requestId = _openOrReuseCycle(CycleType.Fast);

    // Per-cycle cap: total shares in this fast cycle must not exceed fastRedeemCapBps % of TVL.
    if (s_fastRedeemCapBps < BASIS_POINTS) {
        uint256 tvl = (s_sharePrice * i_share.totalSupply()) / SHARE_PRICE_PRECISION;
        uint256 capAssets = (tvl * s_fastRedeemCapBps) / BASIS_POINTS;
        uint256 capShares = (capAssets * SHARE_PRICE_PRECISION) / s_sharePrice;
        if (s_redeemCycles[requestId].shares + _shares > capShares) {
            revert SyVault__FastRedeemCapExceeded();
        }
    }

    s_redeemCycles[requestId].shares             += _shares;
    s_pendingRedeems[_controller][requestId]     += _shares;
    s_redeemRequestOwner[_controller][requestId]  = _owner;
    s_activeRedeemCycleId[_controller]            = requestId;
    s_activeRedeemIsFast[_controller]             = true;
    s_totalLockedShares                           += _shares;

    IERC20(address(i_share)).safeTransferFrom(_owner, address(this), _shares);

    emit FastRedeemRequest(_controller, _owner, requestId, msg.sender, _shares);
}
```

SyVaultRedeem.sol:175-194

```
function cancelFastRedeemRequest(uint256 _requestId, address _controller) external onlyHub {
    uint256 shares = s_pendingRedeems[_controller][_requestId];
    if (shares == 0) revert SyVault__NoPendingRequest();

    RedeemCycle storage cycle = s_redeemCycles[_requestId];
    if (cycle.cycleType != CycleType.Fast)  revert SyVault__NoPendingRequest();
    if (currentEpoch() >= cycle.lockEpoch)  revert SyVault__CycleLocked();

    address owner = s_redeemRequestOwner[_controller][_requestId];

    cycle.shares                              -= shares;
    s_pendingRedeems[_controller][_requestId] = 0;
    s_activeRedeemCycleId[_controller]        = 0;
    s_activeRedeemIsFast[_controller]         = false;
    s_totalLockedShares                       -= shares;

    IERC20(address(i_share)).safeTransfer(owner, shares);

    emit RedeemCancelRequest(_controller, _requestId, shares);
}
```

Attack Scenario:

- Assume total supply is `100e18`

- `s_fastRedeemCapBps` = 1000 (10%), so `capShares` = 10e18

1. A malicious user calls `requestFastRedeem` for `10e18`, fully consuming the cap.

2. Legitimate users attempt to call `requestFastRedeem`, but all transactions revert with `SyVault__FastRedeemCapExceeded`.

3. Once `cutoffEpoch` is crossed (epoch 12), users still cannot submit new fast redeem requests because the previous cycle has not yet been settled (`FastCycleSettlementPending`).

4. Just before `lockEpoch` (epoch 20), the malicious user:

    - Cancels their existing fast redeem request via `cancelFastRedeemRequest`

    - Immediately submits a new `requestFastRedeem` within a single transaction

5. Since the cutoff has already passed and `cycle.shares == 0` after cancellation, `_openOrReuseCycle` creates a new cycle, effectively consuming the cap for the next cycle too.

6. The attacker can repeat this process indefinitely, continuously blocking the fast redeem mechanism for all other users.

Impact:

The fast redeem mechanism can be abused to cause a persistent denial-of-service for legitimate users. Users who rely on operators (e.g. multisigs, portfolio managers, smart contract wallets) to manage time-sensitive redemptions are particularly affected, as the blocked cap could delay critical withdrawals.

Recommendation:

Remove the cancellation feature for fast redeem requests to mitigate this issue.

Resolution: Fixed

Low severity

[L-01] Rate limit applied to gross amount instead of net amount after dust removal

Severity: Low Risk

Description: In `SyOFTAdapter._debit`, the rate limiter's `_outflow` is called with the raw `_amountLD` before `super._debit` processes it. The parent `OFTAdapter._debit` calls `_debitView`, which applies `_removeDust` to strip the fractional portion that cannot be represented on the destination chain. LayerZero OFT uses a shared decimal representation (typically 6 decimals) for cross-chain messages. When the source token has higher precision (e.g. 18 decimals), `_removeDust` truncates the lowest `localDecimals - sharedDecimals` digits so the amount converts cleanly across chains. The actual amount locked in the adapter is this truncated value, not the original `_amountLD`.

Because `_outflow` records the pre-truncation amount, the rate limiter's `amountInFlight` is higher than the real tokens locked. Over many transfers within a single rate limit window, the cumulative over-counting reduces the effective bridgeable capacity.

syOFTAdapter.sol#L66-L74

```
function _debit(
    address _from,
    uint256 _amountLD,
    uint256 _minAmountLD,
    uint32  _dstEid
) internal override returns (uint256 amountSentLD, uint256 amountReceivedLD) {
    _outflow(_dstEid, _amountLD); // @audit gross amount, not net
    return super._debit(_from, _amountLD, _minAmountLD, _dstEid);
}
```

The parent `_debitView` reduces the amount:

```
function _debitView(uint256 _amountLD, ...) internal view virtual returns (uint256 amountSentLD, uint256
amountReceivedLD) {
    amountSentLD = _removeDust(_amountLD); // rounds down
    amountReceivedLD = amountSentLD;
}
```

`_removeDust` strips the digits that cannot survive the local-to-shared decimal conversion:

```
function _removeDust(uint256 _amountLD) internal view virtual returns (uint256 amountLD) {
    return (_amountLD / decimalConversionRate) * decimalConversionRate;
}
```

Where `decimalConversionRate = 10^(localDecimals - sharedDecimals)`. For an 18-decimal token with 6 shared decimals, this is `1e12`.

Impact:

The rate limiter tracks more tokens as "in flight" than are actually locked. Each transfer over-counts by up to `decimalConversionRate - 1` units. For an 18-decimal token bridged to LayerZero's 6 shared decimals, `decimalConversionRate` is `1e12`, so each transfer can over-count by up to `1e12 - 1` wei. While this is small relative to typical transfer sizes, the accounting mismatch means the rate limit is stricter than configured, and users near the limit boundary can be incorrectly blocked.

Recommendation: Apply `_outflow` to the dust-removed amount by calling `_removeDust` before the rate limit check:

```
function _debit(
      address _from,
      uint256 _amountLD,
      uint256 _minAmountLD,
      uint32  _dstEid
  ) internal override returns (uint256 amountSentLD, uint256 amountReceivedLD) {
-     _outflow(_dstEid, _amountLD);
+     _outflow(_dstEid, _removeDust(_amountLD));
      return super._debit(_from, _amountLD, _minAmountLD, _dstEid);
  }
```

Resolution: Fixed

[L-02] Allocator Grace Period Off-by-One: Only 1 Skip Instead of 2 Before Price Override

Severity: Low Risk

Description: The allocator grace period mechanism has an off-by-one error in the skip counter logic. When `setAllocatorActive(true)` is called, the system is designed to skip price updates for `ALLOCATOR_ACTIVE_OVERRIDE_EPOCHS` (= 2) consecutive epochs before forcibly accepting the proposed price. However, due to operator check and incrementation of the `s_allocatorActiveSkips` before the actual check the skips that passes are only one.

```
if (s_allocatorActive) {
    s_allocatorActiveSkips++;                              // ← Increment FIRST
    if (s_allocatorActiveSkips >= ALLOCATOR_ACTIVE_OVERRIDE_EPOCHS) {  // ← Check AFTER with >=
        s_allocatorActive      = false;
        s_allocatorActiveSkips = 0;
        return _proposed;
    }
    return _prev;
}
```

Location: `src/abstracts/SyVaultEpoch.sol:100-114` (function `_resolveEffectivePrice()`)

Acording to the docs: two epoch would be skipped: https://gist.github.com/blckhv/ba075a690fe4f100dee0b9e4c9a4a03a#15-key-rules--invariantshttps://gist.github.com/blckhv/ba075a690fe4f100dee0b9e4c9a4a03a#off-chain-components-and-epoch-system

Impact:

The protocol skips only one epoch instead of intended `ALLOCATOR_ACTIVE_OVERRIDE_EPOCHS = 2`. It returns the new proposed price and the allocator is deactivated on second call intead of skipping 2 epoches.

Recommendation:

Move the increment to occur *after* the threshold check

Or

Replace the `>=` with `>`

Resolution: Fixed

[L-03] Unbounded deposit epoch Iterations make gas costs high for long-term depositors

Severity: Low Risk

Description: Whenever a user calls `requestDeposit()`, a `requestId` is added to the `s_controllerDepositEpochs` array. If the user cancels the deposit request via `cancelDepositRequest()` and then calls `requestDeposit()` again within the same epoch, the same `requestId` is still pushed to the array.

Later, when the user calls `deposit()` or `mint()` after epoch settlement, the function loops from `0` to `s_controllerDepositEpochs.length()` every time, regardless of duplicate or redundant entries.

This results in increasing gas costs for users who frequently deposit assets, as the array grows with each request.

Impact:

Long-term depositors face higher gas costs because every `deposit()` or `mint()` call iterates through the entire `s_controllerDepositEpochs` array

Recommendation: Add a variable `LastSettledEpochIndex` which tracks the last settled epoch index for each `controller` and the next `deposit()`/`mint()` calls should start iterating from this index instead of `0` and set the `LastSettledEpochIndex` at the end of the loop

Resolution: Fixed

[L-04] Price-Zero Cascade: Maintenance Fee Drives Share Price to Zero, Enabling Sentinel Inflation and Delta Bypass

Severity: Low Risk

Description: A combination of three interacting bugs creates a cascading vault compromise when `sharePrice` reaches zero. The root cause is that the maintenance fee calculation in `_collectFees` can reduce the share price to zero when the fee parameters are set to extreme (but accepted) values, and the downstream code fails catastrophically at price zero.

Step 1: Fee drives price to zero. The maintenance fee is computed as:

```
// SyVaultEpoch.sol:L147–L154
if (s_maintenanceFeeInterval > 0 && s_maintenanceFeeBps > 0
    && (_epoch % s_maintenanceFeeInterval) == 0)
{
    uint256 priceBefore  = _price;
    uint256 feeDeduction = (_price * s_maintenanceFeeBps * s_maintenanceFeeInterval)
        / (BASIS_POINTS * EPOCHS_PER_YEAR);
    _price = _price > feeDeduction ? _price - feeDeduction : 0;
```

When `maintenanceFeeBps = 10000` (100%, which passes the `_bps <= BASIS_POINTS` setter validation) and `maintenanceFeeInterval = EPOCHS_PER_YEAR` (8760, which has no upper bound validation), the fee deduction equals the entire price: `feeDeduction = (price * 10000 * 8760) / (10000 * 8760) = price`. The result is `_price = 0`.

Step 2: Sentinel value stored. When `effectivePrice` reaches zero, the epoch settlement stores a sentinel value of 1:

```
// SyVault.sol:L113
s_epochSettlementPrice[_epochToSettle] = _effectivePrice > 0 ? _effectivePrice : 1;
```

Step 3: Massive share inflation. Any pending depositor who claims after this epoch receives astronomically inflated shares because the division uses `settlePrice = 1`:

```
// SyVault.sol:L254
sharesOut += (epochAssets * SHARE_PRICE_PRECISION) / settlePrice;
```

For a 1000-token deposit with 18-decimal precision: `sharesOut = (1000e18 * 1e18) / 1 = 1e39` a 1,000,000,000,000,000,000x inflation factor compared to the expected 1000e18 shares.

Step 4: Delta check bypass. Once `s_sharePrice` reaches zero it stays zero in `SyVaultEpoch`, because `epochSettle` writes the effective price back to `s_sharePrice` at the end of every settlement:

```
// SyVaultEpoch.sol:L82
s_sharePrice = effectivePrice;  // stores 0
```

The sentinel value `1` written by `_onEpochSettle` lives in a completely separate mapping `s_epochSettlementPrice` inside `SyVault`, used only to compute deposit share allocations for that epoch. It never updates `s_sharePrice`.

So on every subsequent call to `epochSettle`, `prevPrice = s_sharePrice = 0`, and the delta guard is skipped:

```
// SyVaultEpoch.sol:L116
if (_prev > 0 && s_maxSharePriceDelta > 0) {  // 0 > 0 → false, skipped entirely
```

This bypass persists for all future epochs until the keeper submits a non-zero price that gets stored back into `s_sharePrice`. Until then, the keeper can set any arbitrary price with no bounds check.

Critically, the delta check in `_resolveEffectivePrice` executes *before* `_collectFees` in the `epochSettle` function flow (line 75-77), so a normal-looking proposed price passes the delta check, and the fee then destroys the price afterward.

Impact:

- All existing shareholders are diluted to near-zero as pending depositors receive 1e18x their expected shares.
- The keeper gains unbounded price authority for all subsequent epochs, enabling direct value extraction through manipulated deposit/redeem rates.
- Combined effect is total vault compromise: complete loss of funds for all existing shareholders.
- The attack requires admin-set fee parameters, but both setters accept the dangerous combination individually `setMaintenanceFeeBps` allows up to 100% and `setMaintenanceFeeInterval` has no upper bound. There is no cross-parameter validation that prevents `maintenanceFeeBps * maintenanceFeeInterval >= BASIS_POINTS * EPOCHS_PER_YEAR`, meaning any combination that satisfies this inequality zeroes the price regardless of how reasonable each parameter looks in isolation.

Recommendation: Apply a multi-layered fix addressing each step of the cascade:

1. Validate fee parameter product. Add a cross-parameter validation to the fee setters ensuring `maintenanceFeeBps * maintenanceFeeInterval < BASIS_POINTS * EPOCHS_PER_YEAR` (i.e., the annualized fee cannot reach 100%):

```
require(
    s_maintenanceFeeBps * s_maintenanceFeeInterval < BASIS_POINTS * EPOCHS_PER_YEAR,
    "fee product exceeds 100%"
);
```

2. Revert on zero price instead of storing sentinel. Replace the sentinel logic with a revert:

```
// SyVault.sol:L113
require(_effectivePrice > 0, "settlement price cannot be zero");
s_epochSettlementPrice[_epochToSettle] = _effectivePrice;
```

3. Remove the `_prev > 0` guard from the delta check so that the check applies unconditionally when `s_maxSharePriceDelta > 0`, or handle the zero-price edge case explicitly.

Resolution: Fixed

[L-05] Fast redeem fee read from storage at claim time instead of request

Severity: Low Risk

Description: The fast redeem fee is read from `s_fastRedeemFeeBps` at claim time in `_claimRedeemAssets` and `withdraw`, not snapshot when the user calls `requestFastRedeem`. If the admin changes `s_fastRedeemFeeBps` while a fast redeem cycle is in progress, users who are past the cancellation cutoff (epoch 12 of a 20-epoch cycle) cannot exit and will be charged the new rate at claim.

`requestFastRedeem` does not store the fee rate - the `RedeemCycle` struct has no fee field:

SyVaultRedeem.sol#L132-L154

```
function requestFastRedeem(uint256 _shares, address _controller, address _owner)
    external onlyHub whenRedeemsNotPaused returns (uint256 requestId)
{
    // ...
    s_redeemCycles[requestId].shares += _shares;
    // no fee snapshot stored
}
```

At claim time, the current storage value is used:

SyVaultRedeem.sol#L362-L367

```
if (cycle.cycleType == CycleType.Fast && s_fastRedeemFeeBps > 0
    && !s_fastRedeemFeeWhitelist[_controller])
{
    fee = (grossAssets * s_fastRedeemFeeBps) / BASIS_POINTS;
}
```

The same pattern appears in `withdraw` at line 268-274.

Impact:

Users who commit to a fast redeem expecting fee X% can be charged fee Y% with no ability to exit. Once past the cutoff epoch, cancellation reverts. For example, if the fee increases from 1% to 5% during the lock period, a user redeeming 1,000 shares worth 1,000 USDC receives 950 USDC instead of the expected 990 USDC, an unexpected 40 USDC loss (4% of assets).

Recommendation: Add a `requestFeeBps` field to the `RedeemCycle` struct and snapshot the fee at request time:

```
struct RedeemCycle {
    CycleType cycleType;
    uint256 shares;
    uint256 cutoffEpoch;
    uint256 lockEpoch;
    bool settled;
    uint256 returnedAmount;
    uint256 cycleRate;
    uint256 requestFeeBps;  // ADD: Fee snapshot
}
```

Update `_openOrReuseCycle()` to snapshot:

```
function _openOrReuseCycle(CycleType _cycleType) internal returns (uint256 cycleId) {
    // ...
    s_redeemCycles[cycleId] = RedeemCycle({
        // ... existing fields
        requestFeeBps: s_fastRedeemFeeBps  // Snapshot current fee
    });
}
```

Update claim logic to use snapshot:

```
if (cycle.cycleType == CycleType.Fast && cycle.requestFeeBps > 0) {
    fee = (grossAssets * cycle.requestFeeBps) / BASIS_POINTS;  // Use snapshot
}
```

Resolution: Fixed

[L-06] First Depositor can inflate `hwm` with low supply resulting in lost `performance fee` for the protocol

Severity: Low Risk

Description: The first depositor of the vault can deposit a dust amount (or the minimum `s_minDepositAmount`) and then directly transfer ERC20 tokens to the liquidity pool, artificially inflating the share price in the next epoch.

This manipulation also increases the `hwm` (high-water mark). However, because the total supply at this stage is very low, the `performance fee` collected is minimal. In contrast, under normal vault operation, reaching a similar `hwm` would require a significantly larger supply, resulting in substantially higher `performance fees`.

As a result, this behavior allows the first depositor to inflate the `hwm` at low supply, leading to a loss of potential `performance fee` revenue for the protocol.

Impact:

First depositor can inflate `hwm` with low supply resulting in lost `performance fee` for the protocol

Recommendation:

Minting a small amount of shares in the `sYVault.sol`'s constructor and transferring them to a dead address can mitigate this issue. This ensures a non-zero initial supply, preventing the first depositor from artificially inflating the `hwm` under low-supply conditions.

Resolution: Acknowledged

[L-07] `requestFastRedeem` Incorrectly Blocked in Deprecated Vault State

Severity: Low Risk

Description: The spec's vault lifecycle table explicitly permits fast redeems in the `deprecated` state, consistent with normal redeems. However, `Hub.requestFastRedeem` uses `_requireActive`, which reverts for any state other than `VaultState.Active`. In contrast, `requestRedeem` correctly uses `_requireActiveOrDeprecated`.

This means users cannot initiate fast redemptions once a vault transitions to `deprecated`, even though the spec intends them to remain available.

src/Hub.sol#L105-L114

```
function requestFastRedeem(
    uint256 _vaultId,
    uint256 _shares,
    address _controller,
    address _owner
) external returns (uint256 requestId) {
    _requireActive(_vaultId);
    _requireOwnerOrOperator(_vaultId, _controller, _owner);
    return ISyVault(_getSyVault(_vaultId)).requestFastRedeem(_shares, _controller, _owner);
}
```

Impact:

When a vault is deprecated, users who prefer the faster 1-day redemption cycle are forced into the slower normal redeem path. This is particularly harmful during vault wind-down scenarios where users want to exit positions quickly. The inconsistency between normal and fast redeem availability in `deprecated` state violates the spec and reduces user optionality at the time they need it most.

Recommendation:

Replace `_requireActive` with `_requireActiveOrDeprecated` in `requestFastRedeem` to match the spec and align with `requestRedeem` behavior.

```
function requestFastRedeem(
      uint256 _vaultId,
      uint256 _shares,
      address _controller,
      address _owner
  ) external returns (uint256 requestId) {
-     _requireActive(_vaultId);
+     _requireActiveOrDeprecated(_vaultId);
      _requireOwnerOrOperator(_vaultId, _controller, _owner);
      return ISyVault(_getSyVault(_vaultId)).requestFastRedeem(_shares, _controller, _owner);
  }
```

Resolution: Fixed

[L-08] `requestRedeemStatus` Returns False `PastCutoff` for Normal Redeem Cycles

Severity: Low Risk

Description: `requestRedeemStatus` returns `PastCutoff` whenever `currentEpoch` is past the current normal cycle's `cutoffEpoch`, without checking whether the cycle is already settled. In contrast, `requestFastRedeemStatus` correctly includes the `!settled` check before returning its blocking status.

More importantly, the `PastCutoff` return is misleading in all past-cutoff cases for normal redeems. `_openOrReuseCycle` for normal cycles unconditionally opens a new cycle when past cutoff, with no settlement check. This means `requestRedeem` would succeed by creating a fresh cycle, but the view function tells callers the request is blocked.

src/syVault.sol#L541-L547

```
function requestRedeemStatus(address _controller) external view returns (RedeemRequestStatus) {
    if (s_pausedFull || s_pausedRedeems) return RedeemRequestStatus.Paused;
    if (s_activeRedeemCycleId[_controller] != 0) return RedeemRequestStatus.AlreadyPending;
    if (s_currentNormalCycleId != 0 && currentEpoch() >=
s_redeemCycles[s_currentNormalCycleId].cutoffEpoch)
        return RedeemRequestStatus.PastCutoff;
    return RedeemRequestStatus.Ok;
}
```

Compare with `requestFastRedeemStatus` which includes `!settled`:

src/syVault.sol#L550-L558

```
function requestFastRedeemStatus(address _controller) external view returns (FastRedeemRequestStatus) {
    if (s_pausedFull || s_pausedRedeems) return FastRedeemRequestStatus.Paused;
    if (s_activeRedeemCycleId[_controller] != 0) return FastRedeemRequestStatus.AlreadyPending;
    if (s_currentFastCycleId != 0
        && currentEpoch() >= s_redeemCycles[s_currentFastCycleId].cutoffEpoch
        && !s_redeemCycles[s_currentFastCycleId].settled)
        return FastRedeemRequestStatus.FastCycleSettlementPending;
    return FastRedeemRequestStatus.Ok;
}
```

The root cause is that `_openOrReuseCycle` for normal cycles opens a new cycle unconditionally when past cutoff, unlike fast cycles which enforce settlement first:

src/abstracts/SyVaultRedeem.sol#L326-L339

```
// Normal — no settlement check, just opens a new cycle
if (s_currentNormalCycleId == 0 || currentEpoch() >= s_redeemCycles[s_currentNormalCycleId].cutoffEpoch) {
    s_currentNormalCycleId = s_cycleIdCounter++;
    // ...
}
```

Impact:

Frontends and integrators relying on `requestRedeemStatus` as a pre-flight check will incorrectly prevent users from submitting normal redeem requests when the current cycle is past cutoff. The actual `requestRedeem` call would succeed by opening a new cycle, but the view function signals otherwise, causing a false negative that blocks legitimate user actions.

Proof of Concept:

1. A normal redeem cycle is created and reaches its `cutoffEpoch`

2. A user (or frontend) calls `requestRedeemStatus` - returns `PastCutoff`

3. Frontend disables the redeem button

4. In reality, calling `requestRedeem` would succeed because `_openOrReuseCycle` opens a new cycle

Recommendation:

At minimum, add the `!settled` check to match `requestFastRedeemStatus` behavior. Beyond that, consider whether `PastCutoff` should ever be returned for normal redeems given that `_openOrReuseCycle` always opens a new cycle past cutoff.

```
function requestRedeemStatus(address _controller) external view returns (RedeemRequestStatus) {
      if (s_pausedFull || s_pausedRedeems) return RedeemRequestStatus.Paused;
      if (s_activeRedeemCycleId[_controller] != 0) return RedeemRequestStatus.AlreadyPending;
-     if (s_currentNormalCycleId != 0 && currentEpoch() >=
s_redeemCycles[s_currentNormalCycleId].cutoffEpoch)
+     if (s_currentNormalCycleId != 0
+         && currentEpoch() >= s_redeemCycles[s_currentNormalCycleId].cutoffEpoch
+         && !s_redeemCycles[s_currentNormalCycleId].settled)
          return RedeemRequestStatus.PastCutoff;
      return RedeemRequestStatus.Ok;
  }
```

Resolution: Fixed

[L-09] `pendingDepositRequest` Returns Non-Zero After Epoch Settlement

Severity: Low Risk

Description: EIP-7540 defines `pendingDepositRequest` and `claimableDepositRequest` as mutually exclusive views into a request's lifecycle. Once an epoch is settled and the request transitions to the Claimable state, `pendingDepositRequest` should return 0.

Currently, `s_pendingDeposits[controller][epoch]` is not zeroed at settlement. Both view functions read from the same storage slot, so after `epochSettle` processes a deposit epoch, both return the same non-zero value for the same `requestId` and `controller`.

src/syVault.sol#L438-L456

```
function pendingDepositRequest(uint256 _requestId, address _controller)
    external view returns (uint256 assets)
{
    return s_pendingDeposits[_controller][_requestId];
}

function claimableDepositRequest(uint256 _requestId, address _controller)
    external view returns (uint256 assets)
{
    if (s_epochSettlementPrice[_requestId] == 0) return 0;
    return s_pendingDeposits[_controller][_requestId];
}
```

`claimableDepositRequest` correctly gates on `s_epochSettlementPrice`, but `pendingDepositRequest` performs no such check and returns the raw storage value regardless of settlement status.

Impact:

Integrators reading `pendingDepositRequest` to determine if a request is still waiting will get stale data. A deposit that has been settled and is ready to claim will still appear as "pending." This breaks the EIP-7540 state machine from an integrator's perspective and can cause downstream logic errors in protocols or frontends that rely on the pending-to-claimable transition being reflected in these view functions.

Recommendation:

Add the settlement price check to `pendingDepositRequest` so it returns 0 once the epoch is settled:

```
function pendingDepositRequest(uint256 _requestId, address _controller)
      external view returns (uint256 assets)
  {
+     if (s_epochSettlementPrice[_requestId] > 0) return 0;
      return s_pendingDeposits[_controller][_requestId];
  }
```

Resolution: Fixed

[L-10] `supportsInterface` Does Not Return True for EIP-165 Interface ID

Severity: Low Risk

Description: EIP-165 requires that any contract implementing `supportsInterface` must return `true` when queried with `0x01ffc9a7` (the EIP-165 interface ID itself). The current implementation only checks for `IERC7540Deposit` and `IERC7540Redeem`, omitting the mandatory self-identification.

src/syVault.sol#L588-L591

```
function supportsInterface(bytes4 _interfaceId) external pure returns (bool) {
    return _interfaceId == type(IERC7540Deposit).interfaceId
        || _interfaceId == type(IERC7540Redeem).interfaceId;
}
```

Impact:

Contracts and tooling that first probe for EIP-165 support via `supportsInterface(0x01ffc9a7)` before querying specific interfaces will receive `false` and skip further checks entirely. This causes the vault to appear as if it does not support any standard interface, breaking standard discovery flows.

Recommendation:

Add the EIP-165 interface ID to the check:

```
function supportsInterface(bytes4 _interfaceId) external pure returns (bool) {
      return _interfaceId == type(IERC7540Deposit).interfaceId
-         || _interfaceId == type(IERC7540Redeem).interfaceId;
+         || _interfaceId == type(IERC7540Redeem).interfaceId
+         || _interfaceId == 0x01ffc9a7;
  }
```

Resolution: Fixed

[L-11] `setAllocator` Does Not Reset `allocatorActive` Flag

Severity: Low Risk

Description: `setAllocator` updates the allocator address but does not reset `s_allocatorActive` or `s_allocatorActiveSkips`:

```
// syVault.sol:351-355
function setAllocator(address _allocator) external onlyAdminGateway {
    emit AllocatorUpdated(s_allocator, _allocator);
    s_allocator = _allocator;
    // s_allocatorActive NOT reset
    // s_allocatorActiveSkips NOT reset
}
```

The `s_allocatorActive` flag is set by the current allocator calling `setAllocatorActive(true)` to signal that it is mid-operation (bridging, deploying capital). If the old allocator had set `s_allocatorActive = true` before being rotated, the new allocator inherits a "locked" state: `epochSettle` will skip the new allocator's proposed price and reuse the previous price for up to `ALLOCATOR_ACTIVE_OVERRIDE_EPOCHS` epochs.

Conversely, if the old allocator had accumulated `s_allocatorActiveSkips`, the new allocator inherits partially-used skip allowance, causing the override timer to fire earlier than expected.

Impact:

(1) If the old allocator left `s_allocatorActive = true`, the new allocator's correct NAV submissions are silently ignored for up to `ALLOCATOR_ACTIVE_OVERRIDE_EPOCHS` epochs, staling the share price. Users deposit and redeem at an incorrect price during this window. (2) If `s_allocatorActiveSkips` is nonzero, the new allocator has fewer epochs before the auto-override fires, which may expose a window where the keeper's potentially incorrect price is accepted before the new allocator can signal its own active state.

Recommendation:

Reset both flags in `setAllocator`:

```
function setAllocator(address _allocator) external onlyAdminGateway {
    emit AllocatorUpdated(s_allocator, _allocator);
    s_allocator         = _allocator;
    s_allocatorActive   = false;   // reset stale flag
    s_allocatorActiveSkips = 0;    // reset skip counter
}
```

Resolution: Fixed

[L-12] Decimal mismatch between syShare and collateral

Severity: Low Risk

Description: `syShare` contract does not override `decimals` function so it has 18 decimals.

However, collateral may have 6 decimals (e.g. USDC)

Considering initial share price is `1e18`, depositing 1e6 USDC will give you 1e6 syShare, which will be rendered in 1e-12 syShare in frontend.

Impact:

Decimal mismatch may cause confusion among users.

Recommendation:

- Match `syShare` decimals to collateral decimal
- Alternatively, normalize deposited asset amount in `syVault` calculation

Resolution: Acknowledged

[L-13] `requestFastRedeemStatus` Returns `Ok` When Remaining Cap Is Below `s_minRedeem`

Severity: Low Risk

Description: The `requestFastRedeemStatus` function checks whether `cycle.shares >= capShares` to decide if the cap has been reached. If the remaining capacity (`capShares - cycle.shares`) is positive but less than `s_minRedeem`, the function returns `FastRedeemRequestStatus.Ok`. However, the actual `requestFastRedeem` call will revert with `SyVault__BelowMinRedeem` because no user can submit a request smaller than `s_minRedeem`.

```
if (s_fastRedeemCapBps < BASIS_POINTS && s_currentFastCycleId != 0) {
     uint256 tvl = (s_sharePrice * i_share.totalSupply()) / SHARE_PRICE_PRECISION;
     uint256 capAssets = (tvl * s_fastRedeemCapBps) / BASIS_POINTS;
     uint256 capShares = (capAssets * SHARE_PRICE_PRECISION) / s_sharePrice;
     if (s_redeemCycles[s_currentFastCycleId].shares >= capShares) {
         return FastRedeemRequestStatus.CapExceeded;
     }
 }
```

This creates a window where the status function signals that a fast redeem is possible, but every attempt to execute one will fail.

Impact:

`requestFastRedeemStatus` returns an incorrect status when `capShares - cycle.shares < s_minRedeem`. Integrating frontends and off-chain systems relying on this pre-flight check will show fast redeems as available when they are effectively blocked, leading to failed transactions and a degraded user experience.

Recommendation:

Account for `s_minRedeem` in the cap check so the status accurately reflects whether a redeemable request can actually be submitted:

```
-   if (s_redeemCycles[s_currentFastCycleId].shares >= capShares) {
-       return FastRedeemRequestStatus.CapExceeded;
-   }
+   if (s_redeemCycles[s_currentFastCycleId].shares + s_minRedeem > capShares) {
+       return FastRedeemRequestStatus.CapExceeded;
+   }
```

Resolution: Fixed

[L-14] `s_minRedeem` can be enforced in `redeem`/`withdraw` functions to prevent users from holding dust amounts in `s_pendingRedeems` and possible dust accumulation in pool

Severity: Low Risk

Description: `s_minRedeem` is enforced in `requestRedeem` and `requestFastRedeem`, but not in `redeem` or `withdraw`.

This inconsistency allows a user to perform a partial redeem that leaves behind a residual share balance below `s_minRedeem` in `s_pendingRedeems`. When the user later attempts to claim this remaining dust balance, the `grossAssets` calculation in `_claimRedeemAssets` may round down to zero (particularly when asset decimals are 6 and share decimals are 18, or during loss cycles where `cycleRate` is low), or the transaction will always revert in `withdraw` because the round-up share calculation produces `shares > pendingShares`.

As a result, the user effectively loses these residual shares without receiving any assets in return when using `redeem`, or their transaction permanently reverts when using `withdraw`, while the corresponding asset value remains in the pool as unclaimable dust inside `s_claimableRedeemAssets`.

The `s_minRedeem` enforcement at request time:

SyVaultRedeem.sol#L95

```
if (s_minRedeem > 0 && _shares < s_minRedeem) revert SyVault__BelowMinRedeem();
```

The `grossAssets` rounding in `_claimRedeemAssets` that truncates dust to zero:

SyVaultRedeem.sol#L362

```
grossAssets = (_shares * cycle.cycleRate) / SHARE_PRICE_PRECISION;
```

The round-up share calculation in `withdraw` that causes reverts on dust:

SyVaultRedeem.sol#L278-L281

```
shares = cycle.cycleRate > 0
    ? (grossAssets * SHARE_PRICE_PRECISION + cycle.cycleRate - 1) / cycle.cycleRate
    : 0;
if (shares == 0 || shares > pendingShares) revert SyVault__NoClaimableRedeem();
```

Impact:

Accumulation of dust assets in the pool. Users who perform partial redeems that leave a sub-`s_minRedeem` residual will either receive zero assets (via `redeem`) or face permanently reverting transactions (via `withdraw`), with the corresponding value locked in `s_claimableRedeemAssets`.

Recommendation:

Add a dust-sweep check in `redeem` after updating `s_pendingRedeems`. If the remaining balance falls below `s_minRedeem`, set `_shares` to the full `pendingShares` to prevent sub-minimum residuals:

```
uint256 pendingShares = s_pendingRedeems[_controller][s_activeRedeemCycleId[_controller]];
if (pendingShares - _shares > 0 && pendingShares - _shares < s_minRedeem) {
    _shares = pendingShares;
}
```

Add the same check in `withdraw`. If `pendingShares - shares < s_minRedeem`, set `shares = pendingShares`, then recompute `grossAssets` from the adjusted shares using `_claimRedeemAssets` (which applies the fee via `cycleRate` and `feeBps`) instead of directly subtracting `_assets`, and transfer the full corresponding net assets to the user.

Resolution: Fixed

[L-15] HWM stale when performance fee is zero, retroactive fee collection - Formal Verification

Severity: Low Risk

Description:

The high-water mark (`s_hwm`) is only updated inside the `if (_price > hwm && s_performanceFeeBps > 0)` block in `_collectFees()` (SyVaultEpoch.sol:129). When `s_performanceFeeBps == 0`, neither the fee collection NOR the HWM update occurs, even if the share price grows above the old HWM.

The `setPerformanceFeeBps()` setter (SyVault.sol:392-396) updates the fee rate without syncing `s_hwm` to the current `s_sharePrice`. This creates a gap: after re-enabling the fee, the first `epochSettle` call sees `effectivePrice > s_hwm` (the stale, pre-fee-off HWM) and collects performance fee on the entire difference — including gains that occurred while the fee was zero.

Example: HWM = 1.00, fee set to 0, share price grows to 1.50 over 100 epochs, fee re-enabled at 500 bps. Next `epochSettle(1.55)` charges 500 bps on (1.55 - 1.00) = 0.55 instead of just (1.55 - 1.50) = 0.05. Shareholders are charged 11x the legitimate fee.

Recommendation:

The retroactive fee collection path is real and mechanically executable. The fix should sync `s_hwm` to `s_sharePrice` in `setPerformanceFeeBps()` when transitioning from 0 to non-zero:

```
function setPerformanceFeeBps(uint256 _bps) external onlyAdminGateway {
    if (_bps > BASIS_POINTS) revert SyVault__InvalidParam();
    // Sync HWM when re-enabling performance fee to prevent retroactive collection
    if (s_performanceFeeBps == 0 && _bps > 0) {
        s_hwm = s_sharePrice;
    }
    emit PerformanceFeeBpsUpdated(s_performanceFeeBps, _bps);
    s_performanceFeeBps = _bps;
}
```

Resolution: Fixed

Informational severity

[I-01] Redeem status views defined in `syVault` instead of `SyVaultRedeem`

Severity: Information Risk

Description: `requestRedeemStatus` and `requestFastRedeemStatus` are defined in `syVault.sol` but operate entirely on state variables and types from `SyVaultRedeem` (`s_activeRedeemCycleId`, `s_currentNormalCycleId`, `s_currentFastCycleId`, `s_redeemCycles`, `CycleType`). The actual request functions they mirror, `requestRedeem` and `requestFastRedeem`, along with `_openOrReuseCycle`, all live in `SyVaultRedeem`.

syVault.sol#L547-L563

```
// In syVault.sol — but all referenced state lives in SyVaultRedeem
function requestRedeemStatus(address _controller) external view returns (RedeemRequestStatus) {
    if (s_pausedFull || s_pausedRedeems) return RedeemRequestStatus.Paused;
    if (s_activeRedeemCycleId[_controller] != 0) return RedeemRequestStatus.AlreadyPending;
    if (s_currentNormalCycleId != 0 && currentEpoch() >=
s_redeemCycles[s_currentNormalCycleId].cutoffEpoch)
        return RedeemRequestStatus.PastCutoff;
    return RedeemRequestStatus.Ok;
}

function requestFastRedeemStatus(address _controller) external view returns (FastRedeemRequestStatus) {
    if (s_pausedFull || s_pausedRedeems) return FastRedeemRequestStatus.Paused;
    if (s_activeRedeemCycleId[_controller] != 0) return FastRedeemRequestStatus.AlreadyPending;
    if (s_currentFastCycleId != 0
        && currentEpoch() >= s_redeemCycles[s_currentFastCycleId].cutoffEpoch
        && !s_redeemCycles[s_currentFastCycleId].settled)
        return FastRedeemRequestStatus.FastCycleSettlementPending;
    return FastRedeemRequestStatus.Ok;
}
```

Impact:

Separating the status views from the logic they describe makes it harder to keep them in sync.

Recommendation:

Move `requestRedeemStatus` and `requestFastRedeemStatus` into `SyVaultRedeem` alongside `requestRedeem`, `requestFastRedeem`, and `_openOrReuseCycle` so that redeem eligibility logic is co-located with the enforcement it describes.

Resolution: Fixed

[I-02] Misleading error name in cancel redeem functions

Severity: Information Risk

Description: `cancelRedeemRequest` and `cancelFastRedeemRequest` revert with `SyVault__CycleNotLocked` when the cycle **is** locked, which is the opposite of what the error name conveys. The condition `currentEpoch() >= cycle.lockEpoch` evaluates to true when the lock epoch has been reached, meaning the cycle is locked and cancellation is no longer allowed.

SyVaultRedeem.sol#L115

```
function cancelRedeemRequest(uint256 _requestId, address _controller) external onlyHub {
    // ...
    if (currentEpoch() >= cycle.lockEpoch) revert SyVault__CycleNotLocked(); // cycle IS locked
}
```

SyVaultRedeem.sol#L163

```
function cancelFastRedeemRequest(uint256 _requestId, address _controller) external onlyHub {
    // ...
    if (currentEpoch() >= cycle.lockEpoch) revert SyVault__CycleNotLocked(); // cycle IS locked
}
```

The same error is used correctly in `settleRedeemBatch` at line 187, where it reverts when `currentEpoch() < cycle.lockEpoch` (the cycle is genuinely not yet locked).

Impact:

Misleading error name makes debugging harder for integrators. Off-chain tooling parsing revert reasons would interpret the error as "cycle is not locked" when the actual cause is the opposite.

Recommendation:

Rename the error to `SyVault__CycleLocked` in the cancel functions, or introduce a separate error for clarity while keeping `SyVault__CycleNotLocked` for `settleRedeemBatch`.

Resolution: Fixed

[I-03] Incorrect NatSpec for `s_lastFeeCollectionEpoch`

Severity: Information Risk

Description: The NatSpec for `s_lastFeeCollectionEpoch` states it tracks the last epoch at which "performance or maintenance fee" was collected. In practice, it is only updated inside the maintenance fee block and tracks maintenance fee collection exclusively.

src/abstracts/SyVaultEpoch.sol#L48-L49

```
/// @dev Last epoch at which performance or maintenance fee was collected.
uint256 internal s_lastFeeCollectionEpoch;
```

The variable is only written at line 155, inside the maintenance fee branch:

src/abstracts/SyVaultEpoch.sol#L141-L156

```
// ── Maintenance fee ──
if (s_maintenanceFeeInterval > 0 && s_maintenanceFeeBps > 0
    && (_epoch % s_maintenanceFeeInterval) == 0)
{
    // ... maintenance fee logic ...
    s_lastFeeCollectionEpoch = _epoch;
}
```

The performance fee branch does not update it. This is correct behavior since the variable is interval-gated and only relevant to maintenance fee scheduling. The NatSpec is what is wrong.

Impact:

No on-chain impact. The variable is never read on-chain. Misleading documentation may confuse developers or auditors into thinking the variable should also track performance fee epochs.

Recommendation: Update the NatSpec to reflect actual usage.

Resolution: Fixed

[I-04] `setMaxTvl` Should Not Allow Setting `s_maxTVL` Below the `Current TVL`

Severity: Information Risk

Description:

`setMaxTvl()` function:

```
function setMaxTvl(uint256 _maxTVL) external onlyAdminGateway {
    emit MaxTvlUpdated(s_maxTVL, _maxTVL);
    s_maxTVL = _maxTVL;
}
```

The `setMaxTvl()` function should validate that the `_maxTVL` passed is not lower than the `current TVL` and if it is then the function should revert.

Impact:

Setting `maxTVL` below `current TVL` will cause `requestDeposit()` calls to revert

Recommendation:

Add the following check to the `setMaxTvl()` function

```
function setMaxTvl(uint256 _maxTVL) external onlyAdminGateway {
+     if (_maxTVL < (s_sharePrice * i_share.totalSupply()) / SHARE_PRICE_PRECISION) revert();
        emit MaxTvlUpdated(s_maxTVL, _maxTVL);
        s_maxTVL = _maxTVL;
    }
```

Resolution: Acknowledged

[I-05] `set_fast_redeem_fee_bps` allows 100% fee, leading to division by zero in `withdraw` and total fund loss in `redeem`

Severity: Information Risk

Description: The Solidity `setFastRedeemFeeBps` setter uses `>` instead of `>=`, allowing `_bps = 10000` (100% fee):

```
// syVault.sol:486-490
function setFastRedeemFeeBps(uint256 _bps) external onlyAdminGateway {
    if (_bps > BASIS_POINTS) revert SyVault__InvalidParam();  // allows 10000
    emit FastRedeemFeeBpsUpdated(s_fastRedeemFeeBps, _bps);
    s_fastRedeemFeeBps = _bps;
}
```

This produces identical failures to the Cairo version:

- **`withdraw()`** calls `_netToGrossAssets` which divides by `BASIS_POINTS - feeBps = 0` (`SyVaultRedeem.sol:285`) — division by zero revert
- **`redeem()`** computes `fee = gross`, returning 0 net assets (`SyVaultRedeem.sol:392`) — shares already burned at settlement, total loss

The protocol team confirmed 100% fee has no legitimate use case. The agreed-upon maximum reasonable fee is 30% (3000 bps).

Recommendation:

```
+ uint256 public constant MAX_FAST_REDEEM_FEE_BPS = 3000;  // 30% maximum

  function setFastRedeemFeeBps(uint256 _bps) external onlyAdminGateway {
-     if (_bps > BASIS_POINTS) revert SyVault__InvalidParam();
+     if (_bps > MAX_FAST_REDEEM_FEE_BPS) revert SyVault__InvalidParam();
      emit FastRedeemFeeBpsUpdated(s_fastRedeemFeeBps, _bps);
      s_fastRedeemFeeBps = _bps;
  }
```

Resolution: Fixed

[I-06] Deposit epoch consumption loops use `continue` instead of `break` on unsettled epochs

Severity: Information Risk

Description: `_consumeDepositEpochsByAssets` and `_consumeDepositEpochsByShares` use `continue` when encountering an unsettled epoch (`settlePrice == 0`), allowing the loop to iterate past it and process newer epochs.

In practice, epochs are settled sequentially, so a newer epoch can never have a non-zero `settlement_price` while an older one remains unsettled. The `continue` is functionally equivalent to `break`, but wastes gas by iterating over all remaining (guaranteed unsettled) entries.

SyVault.sol:240

```
function _consumeDepositEpochsByAssets(address _controller, uint256 _assets)
    internal
    returns (uint256 sharesOut)
{
    uint256[] storage epochs = s_controllerDepositEpochs[_controller];
    uint256 len              = epochs.length;
    uint256 remaining        = _assets;

    for (uint256 i; i < len; ++i) {
        uint256 epoch       = epochs[i];
        uint256 settlePrice = s_epochSettlementPrice[epoch];
        if (settlePrice == 0) continue; // @audit could break early
        // ...
    }
}
```

SyVault.sol:272

```
function _consumeDepositEpochsByShares(address _controller, uint256 _sharesTarget)
    internal
    returns (uint256 assetsOut)
{
    uint256[] storage epochs = s_controllerDepositEpochs[_controller];
    uint256 len              = epochs.length;
    uint256 sharesRemaining  = _sharesTarget;

    for (uint256 i; i < len; ++i) {
        uint256 epoch       = epochs[i];
        uint256 settlePrice = s_epochSettlementPrice[epoch];
        if (settlePrice == 0) continue; // @audit could break early
        // ...
    }
}
```

Impact:

Users with many unsettled epochs in their `s_controllerDepositEpochs` array pay unnecessary gas for no-op loop iterations.

Recommendation:

Replace `continue` with `break` in both functions to exit the loop as soon as an unsettled epoch is reached.

Resolution: Fixed

[I-07] `claimableFastRedeemRequest` Returns An `_assets` Value That `withdraw` Cannot Accept

Severity: Information Risk

Description: `claimableFastRedeemRequest` computes the claimable net amount using floor rounding, while `withdraw` rounds the inverse → gross conversion up. Because of this asymmetry, the number returned by the view cannot in general be passed back into `withdraw` without reverting.

The view computes:

```
gross_view  = floor(shares * cycle_rate / SHARE_PRICE_PRECISION)
fee_view    = floor(gross_view * fee_bps / BASIS_POINTS)
net_view    = gross_view - fee_view
```

`withdraw` takes `_assets` as the net the controller wants and computes:

```
grossAssets  = ceil(_assets * BASIS_POINTS / (BASIS_POINTS - feeBps))
shares       = ceil(grossAssets * SHARE_PRICE_PRECISION / cycleRate)
if (shares == 0 || shares > pendingShares) revert SyVault__NoClaimableRedeem();
```

When `gross_view * feeBps` is not a multiple of `BASIS_POINTS`, the inverse conversion applied to `net_view` returns `gross_view + 1`. That extra unit forces `shares` to exceed `pendingShares` and the call reverts with `SyVault__NoClaimableRedeem()`.

SyVaultRedeem.sol#L441-L454

```
function claimableFastRedeemRequest(uint256 _requestId, address _controller)
    external
    view
    returns (uint256 assets)
{
    RedeemCycle storage cycle = s_redeemCycles[_requestId];
    if (cycle.cycleType != CycleType.Fast || !cycle.settled) return 0;
    uint256 shares = s_pendingRedeems[_controller][_requestId];
    if (shares == 0) return 0;
    uint256 gross = (shares * cycle.cycleRate) / SHARE_PRICE_PRECISION;
    if (s_fastRedeemFeeWhitelist[_controller]) return gross;
    return gross - (gross * cycle.feeBps / BASIS_POINTS);
}
```

SyVaultRedeem.sol#L280-L295

```
if (cycle.cycleType == CycleType.Fast && cycle.feeBps > 0
    && !s_fastRedeemFeeWhitelist[_controller])
{
    grossAssets = (_assets * BASIS_POINTS + BASIS_POINTS - cycle.feeBps - 1)
        / (BASIS_POINTS - cycle.feeBps);
    fee = grossAssets - _assets;
} else {
    grossAssets = _assets;
}

shares = cycle.cycleRate > 0
    ? (grossAssets * SHARE_PRICE_PRECISION + cycle.cycleRate - 1) / cycle.cycleRate
    : 0;
if (shares == 0 || shares > pendingShares) revert SyVault__NoClaimableRedeem();
```

Impact:

EIP-7540 and EIP-4626 require that a user can always withdraw up to the value reported by the claimable view. Here `claimableFastRedeemRequest` advertises a maximum that `withdraw` rejects in almost every non-trivial case. Integrations that forward the view output directly to `withdraw` will fail, and users following frontend hints will see their transactions revert. The shares remain claimable through `redeem`, so no funds are lost, but the claimable view effectively overstates what the exact-fill `withdraw` path can honor.

Proof of Concept:

With `feeBps = 3000` (30%) and `pendingShares` such that `gross_view = 101`:

1. `fee_view = (101 * 3000) / 10000 = 30`
2. `net_view = 101 - 30 = 71`
3. User reads `71` from `claimableFastRedeemRequest` and calls `withdraw` with `_assets = 71`
4. `withdraw` computes `grossAssets = (71 * 10000 + 10000 - 3000 - 1) / 7000 = 102`
5. `shares = ceil(102 * SHARE_PRICE_PRECISION / cycleRate) > pendingShares`
6. Call reverts with `SyVault__NoClaimableRedeem()`

Recommendation:

Align the view with the rounding used by `withdraw` so that the advertised value is always reachable. Compute the same `gross` the current view produces and then return the largest `_assets` such that the inverse `net→gross` conversion is less than or equal to `gross`:

```
function claimableFastRedeemRequest(uint256 _requestId, address _controller)
    external
    view
    returns (uint256 assets)
{
    RedeemCycle storage cycle = s_redeemCycles[_requestId];
    if (cycle.cycleType != CycleType.Fast || !cycle.settled) return 0;
    uint256 shares = s_pendingRedeems[_controller][_requestId];
    if (shares == 0) return 0;
    uint256 gross = (shares * cycle.cycleRate) / SHARE_PRICE_PRECISION;
    if (s_fastRedeemFeeWhitelist[_controller]) return gross;
    // Largest net such that net_to_gross(net) <= gross.
    return (gross * (BASIS_POINTS - cycle.feeBps)) / BASIS_POINTS;
}
```

An equivalent and smaller change is to return `net - 1` whenever a fast fee is applied and `net > 0`, which guarantees reachability at the cost of one wei of precision.

Resolution: Fixed